

Lista de Exercícios –

Structs e Unions

1) Defina um tipo estruturado que represente um paciente dentro do sistema de saúde e armazene as seguintes informações: nome, idade, tipo sanguíneo e uma lista com o nome de doenças que possa ter. Use bit fields para diminuir o tamanho da estrutura. Crie um programa que permita ao usuário cadastrar pacientes e exiba as informações cadastradas.

2) Amplie a funcionalidade do programa do exercício anterior, de forma que seja realizada a busca das informações do paciente com base no seu nome. Se existir mais de um paciente com o mesmo nome, o programa deve apresentar os dados de ambos.

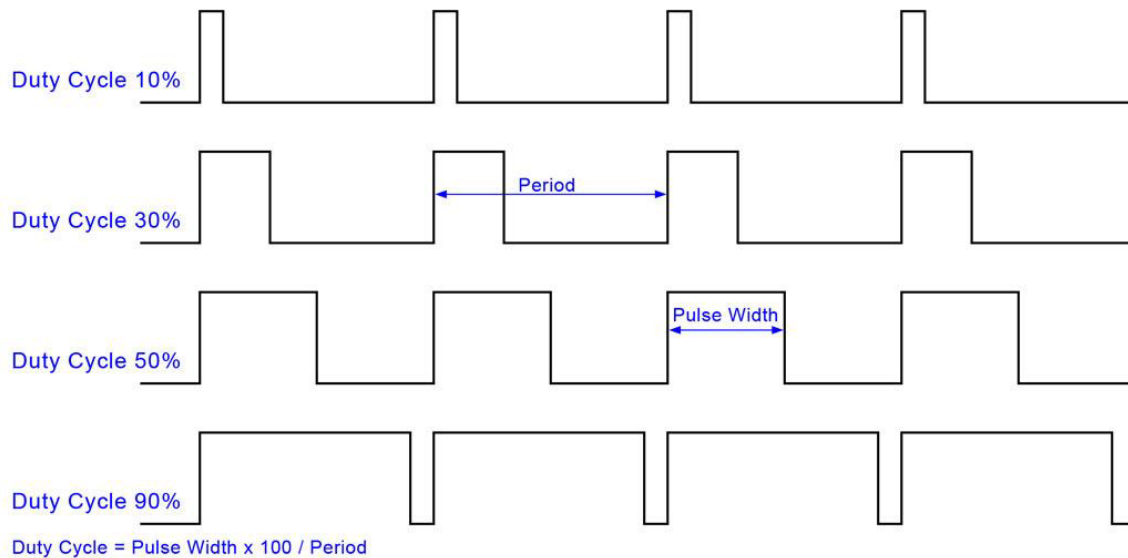
3) Estamos interessados em guardar informações sobre diversas moléculas químicas. Para tanto, cada molécula será representada por um tipo estruturado armazenados em um vetor. Cada estrutura deve possuir as seguintes informações das moléculas:

- Nome da molécula;
- Massa molecular (em precisão dupla);
- Vetor de átomos, onde cada elemento do vetor contém o nome dos elementos em que ele se liga na molécula;
- Densidade (em precisão dupla);
- Ponto de fusão (em precisão dupla) e;
- Ponto de ebulição (em precisão dupla)

Seu programa deve ser capaz de pedir as informações de uma molécula e armazenar. Também, deve ser capaz de retornar os dados das moléculas requisitadas, se passados qualquer um dos dados disponíveis no banco de dados.

4) Modulação por largura de pulso (*pulse width modulation* - PWM) é uma técnica para controlar a potência de uma carga apenas com 0 e 1 lógicos. Um sinal de tensão periódico é aplicado a uma carga, variando entre 0 (0 V) e 1 (uma tensão máxima). Quanto mais tempo o sinal de tensão que controla a carga tiver em nível lógico 0 ou 1 (depende do sinal), maior será a potência transferida para a carga. Chamamos a quantidade de tempo em que o sinal fica no nível lógico desejado como ciclo de trabalho (*duty cycle*) sendo usualmente representada em percentual. Crie uma estrutura que possua: a frequência do

sinal em Hertz; o *duty cycle* em percentual e; o nível lógico do *duty cycle*, que pode ser tanto 0 ou 1. Faça o *duty cycle* como bit field de um 1 bit, de forma que a struct como um todo seja múltipla de 8 bits.



- 5) Crie uma classe que receba no seu construtor a struct criada no exercício anterior. A classe deve ser capaz de mudar os campos da struct com o uso de funções *setter* e retornar seus parâmetros com o uso de funções *getter*.
- 6) Refaça o exercício anterior, porém implemente os métodos da classe criada dentro da própria struct e desconsidere o uso de classe.
- 7) Defina uma union que contenha um inteiro, um caractere e um float. Escreva um programa que demonstre como acessar cada membro da union.
- 8) Crie um programa que declare uma union contendo vários tipos de dados e imprima o tamanho da union usando `sizeof`. Compare o tamanho da union com o tamanho dos tipos de dados individuais que a compõem.
- 9) Desenvolva uma função que aceite uma union como argumento e imprima o valor de cada membro.
- 10) Cores RGB podem ser representadas por um único valor inteiro de 32 bits, ou por uma tripla de 8 bits cada para as cores vermelha, verde e azul. Crie um objeto contendo uma union "rgbColor" que possa tanto representar uma cor por meio de um valor inteiro de 32

bits, ou por uma tripla de 8 bits cada. O seu programa deve ser capaz de ler os valores r, g e b do *prompt* e imprimir o valor de 32 bits correspondente.

11) Queremos que um objeto represente uma forma geométrica que assume diferentes formatos ao longo da execução do programa. As possíveis formas são: círculo (definido pelo seu raio), retângulo (definido pela altura e largura) e triângulo (definido pelo comprimento de cada um de seus lados). Sua classe deve possuir funções *getter* e *setter* para cada uma das formas. Cada vez que for dado um *set* para definir uma propriedade de uma forma (exemplo: `SetCirculo(float raio)`), será retornada a área com *getter* da última forma setada (exemplo: `float Area(void)`). A classe terá um *setter* para cada forma.

12) Para cada struct abaixo, verifique se a mesma está otimizada para ocupar memória, e caso não esteja, como ajustar para que ocupe menos memória. Explique porque o ajuste fez com que o uso de memória tenha sido otimizado e como o arranjo de bytes da struct ficou na memória.

a)

```
struct A
{
    char c;
    int x;
    short s;
};
```

b)

```
struct A
{
    int x;
    short s;
    unsigned y;
    char c;
};
```

c)

```
struct C
{
    int x : 6;
    short s : 2;
    long y : 4;
    char c : 3;
    unsigned z;
};
```

d)

```
struct A
{
    char str[5];
    long y;
    long long x;
    int z : 3;
};
```